WORKSHOP LISTES CHAINÉES

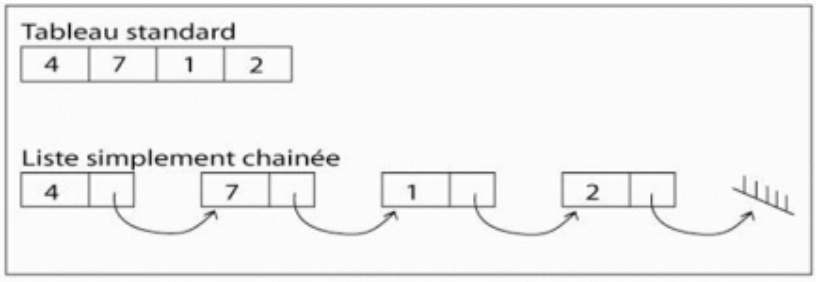
NOTION DE STRUCTURE DE DONNÉES : LISTES CHAINÉES

Définition

Une liste chainée est une structure de données à accès séquentiel :

- Une liste chaînée représente un ensemble d'éléments rangés linéairement mais pas forcément contigus en mémoire
- Contrairement au tableau, l'accès se fait par des pointeurs
- Chaque élément, appelé une cellule, contient un ensemble d'informations dont un ou plusieurs liens vers d'autres éléments de la liste.
- Les liens (pointeurs) qui désignent des éléments particuliers (ex. premier et/ou dernier) sont stockés à part.

Dans la figure ci-dessous une présentation d'un tableau et une liste simplement chainée :



Il existe plusieurs types de listes chaînées dont les 4 classiques :

- La liste simplement chaînée
- La liste simplement chaînée circulaire
- La liste doublement chaînée
- La liste doublement chaînée circulaire

Tout au long de ce workshop nous allons aborder que les listes simplement chainées.

Opérations possibles sur une liste :

- Créer une liste chainée
- Détruire une liste chainée
- Vider une liste chainée
- Récupérer le premier élément, le dernier ou l'élément courant
- Ajouter un élément avant le premier élément de la liste (Ajouter en tête)
- Ajouter un élément après le dernier élément de la liste (Ajouter en queue)
- Insérer un élément avant l'élément courant
- Supprimer l'élément courant
- Supprimer un élément en tête de liste,
- Supprimer un élément à la fin de la liste.
- Etc.

Elément d'une liste chainée

Un élément d'une liste chainée est défini par une structure. Cette structure contient les informations en relation avec les objectifs de l'application et un pointeur sur une structure de même type :

```
1 typedef struct element
2 {
3     Type data;
4     struct element *suivant;
5 }element;
```

Représenter une liste chainée

Grâce au pointeur chaque élément peut contenir l'adresse d'un élément et une liste chainée peut se représenter ainsi :

WORKSHOP LISTES CHAINÉES



Chaque carré correspond à un élément de la liste. Le petit carré noir en haut à gauche correspond à l'adresse en mémoire de l'élément. Le petit rond en bas à droite correspond au pointeur qui "pointe" sur le suivant, c'est à dire qui contient l'adresse de l'élément suivant.

Au départ, il y a le pointeur de tête qui contient l'adresse du premier élément c'est-à-dire l'adresse de la liste.

Le début d'une liste chainée

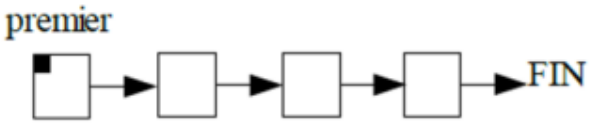
Deux positions sont très importantes dans une liste chainée : le début et la fin, souvent désignées par "premier et dernier" ou "tête et queue". Sans le premier impossible de savoir où commence la liste et sans le dernier impossible de savoir où elle s'arrête.

L'adresse de la liste est donnée par le premier élément, celui à partir duquel tous les autres sont accessibles. En général une liste prend ainsi le nom du premier élément. Ce premier élément est un pointeur que vous déclarerez dans votre programme principal ainsi :

```
1 element *premier = NULL ;
```

Parcourir une liste chainée

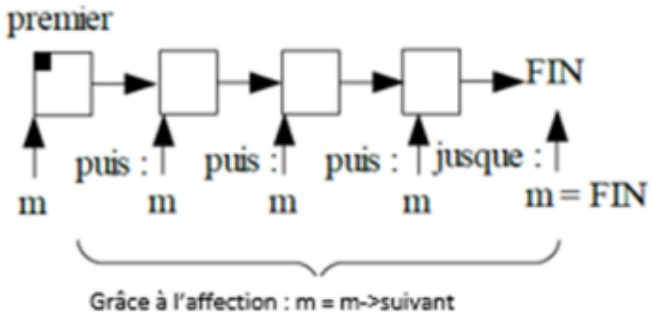
Supposons qu'on a une liste déjà constituée de quatre éléments :



Pour parcourir la liste nous allons faire avec les adresses de chaque élément : passer d'un élément à l'autre grâce à un pointeur intermédiaire, ce qui donne par exemple :

Au départ :

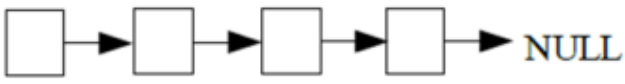
```
1 m= premier
```



Marquer la fin d'une liste chainée

Il y a plusieurs possibilités pour marquer la fin de la liste:

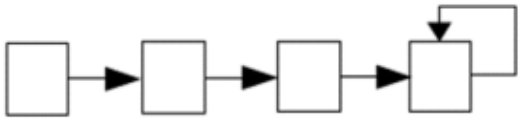
Fin de la liste lorsque l'adresse de l'élément suivant vaut NULL :



La fin est connue si :

```
1 m->suivant==NULL
```

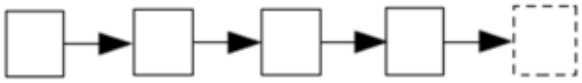
L'adresse de l'élément suivant est l'adresse de l'élément lui-même :



La fin est connue si :

WORKSHOP LISTES CHAINÉES

La liste est définie par un pointeur de début et un pointeur de fin. Le pointeur de fin désigne toujours le dernier élément de la liste et il sert de sentinelle :



Le dernier

La fin est connue si :

```
m->suivant==dernier
```